

Stratégie d'Optimisation à Grande Échelle

Maîtrise des coûts LLM et d'infrastructure pour 500k utilisateurs

1. Le Paradigme du "Global Pool" (Mutualisation du Scraping)

Pour absorber 500 000 utilisateurs sans faire exploser les coûts de proxy et de compute, l'architecture doit abandonner le modèle "1 utilisateur = 1 recherche". Le système doit être inversé :

- **Workers Centralisés** : Les instances Playwright Stealth tournent en continu pour scraper les plateformes (LinkedIn, Indeed) par secteurs géographiques et mots-clés globaux.
- **Base de données unique** : Les offres nettoyées sont ingérées dans **PostgreSQL**. Lorsqu'un utilisateur effectue une recherche, il interroge cette base mutualisée, ce qui réduit la charge de scraping de 99%.

Le module backend (Fastify) interroge d'abord la base de données. Si le pool est vide pour un secteur de niche, un événement est poussé dans BullMQ pour déclencher un worker de scraping ciblé de manière asynchrone.

2. L'Entonnoir de Filtrage Vectoriel (Remplacement du LLM)

Le LLM ne doit jamais être utilisé pour trier ou chercher des offres, son coût temporel et financier est prohibitif. La stratégie repose sur la recherche sémantique :

- **Génération d'Embeddings** : À l'inscription, le CV de l'utilisateur est converti en vecteur. Chaque nouvelle offre d'emploi subit le même traitement.
- **Base de Données Vectorielle** : Utilisation de **Qdrant** pour stocker ces millions de vecteurs.
- **Microservices Haute Performance** : Pour supporter le calcul de similarité cosinus de 500k utilisateurs, un microservice dédié développé en **Go** ou **Rust** interroge Qdrant. L'opération prend quelques millisecondes et ne coûte pratiquement rien en ressources.

3. Auto-hébergement et Optimisation de l'Inférence (vLLM)

La facturation au token via des API tierces est le principal danger financier du projet. La solution est le déploiement de modèles open-source sur des clusters GPU dédiés.

Kubernetes

Docker

vLLM

Qwen 2.5

Llama 3

- **Continuous Batching** : En utilisant vLLM sur des instances GPU nues (ex: RunPod, Hetzner), le serveur traite des dizaines de requêtes d'évaluation simultanément. Cela maximise l'utilisation de la VRAM et divise le coût par requête par un facteur de 10 à 50.
- **Routage de Modèles** : Un modèle ultra-rapide (ex: Qwen 7B) est dédié au scoring JSON (Oui/Non/Note). Un modèle plus lourd n'est sollicité que pour la génération finale (rédaction de lettre).

4. Mise en Cache Agressive (Redis)

Dans un contexte de recherche d'emploi, des milliers d'utilisateurs cibleront les mêmes offres populaires. L'architecture doit capitaliser là-dessus :

- **Hachage et Cache** : Le hash de l'offre et l'empreinte du profil utilisateur servent de clé dans **Redis**. Si une offre a déjà été analysée et scorée pour un profil similaire (ex: Développeur Next.js Senior avec 5 ans d'expérience), le résultat LLM est récupéré depuis le cache.
- Cela réduit drastiquement les appels redondants aux GPU.

5. Aligner l'Infrastructure sur le Business (Freemium)

L'optimisation technique doit être couplée à une logique produit rigoureuse pour garantir la viabilité :

Plan Gratuit : Accès uniquement au matching vectoriel (Qdrant) et aux filtres classiques (PostgreSQL). Le coût marginal par utilisateur est proche de zéro.

Plan Premium : Débloque l'agent IA. Les requêtes de scoring LLM asynchrones (via BullMQ) et la génération de lettres de motivation personnalisées justifient le coût de l'abonnement SaaS, finançant ainsi les instances GPU.